Lesson Activities — 2.1 Elements of Computational Thinking

Hands-on classroom activities for OCR H446 Section 2.1 — students practise *applying* the five thinking skills to concrete scenarios, since the exam tests application to unseen contexts, not recited definitions.

2.1.1 Thinking Abstractly

Map to the Canteen (abstraction map-drawing · 20 min · individuals, then pairs)

Resources: Plain paper; pencils; (optional) a printed satellite photo or floor plan of the school for the reveal. **How it runs:** 1. Without discussion, each student draws directions from this classroom to the canteen for a brand-new Year 7 who has never seen the building. 5 minutes, no rules given about style. 2. Pair up and swap. Partners "walk" the route mentally and list: what details are *on* the map, and what real-world details were *left out* (wall colours, door widths, number of windows, exact distances, other rooms passed). 3. Class harvest on the board in two columns — **kept** (turns, landmarks, ordering, decision points like "at the stairs…") vs **discarded** — then the key question: *why* was each discarded detail safe to remove? (Because it doesn't change whether the Year 7 arrives.) 4. Introduce the definition *after* the experience: abstraction = removing/hiding unnecessary detail so only information relevant to the problem remains. Show the satellite photo: "Is my map *wrong* because it doesn't match this? No — it serves a different purpose." 5. Twist round: redraw the map for a *different purpose* — a wheelchair user, or a fire-evacuation plan. What must now be added back in (lifts/ramps; all exits) and what can now go? **Answers / expected outcomes:** Kept: sequence of moves, landmarks, decision points. Discarded: appearance, scale, irrelevant rooms — justified as irrelevant *to this problem*. The twist proves the crucial exam point: what counts as "necessary detail" depends on the problem, so a good answer always justifies kept/discarded relative to the stated purpose. Link forward: the London Underground map, road networks as graphs of nodes and weighted edges. **Stretch:** Students name what their map's *interface* is (the instructions the Year 7 follows) versus its hidden *implementation* (the actual building), and write one sentence connecting this to a variable being an abstraction of a memory address.

Representational or Generalisation? (card sort · 15 min · pairs)

Resources: 10 example cards per pair; two header cards — **Representational abstraction** (remove detail from one specific thing) and **Generalisation/categorisation** (group many things by shared characteristics). **How it runs:** 1. Pairs sort the cards: (a) the Tube map; (b) an `Animal` superclass for Dog, Cat, Horse; (c) a road network stored as a graph of nodes and weighted edges; (d) a `Vehicle` record used for cars, vans and bikes in a rental system; (e) a weather model ignoring individual raindrops; (f) grouping savings and current accounts under one `Account` type; (g) a game minimap; (h) a `Shape` class with `area()` overridden by Circle and Square; (i) a flight simulator's simplified physics; (j) treating all customers as `name, ID, balance`. 2. Pairs must write the *test question* they used to decide (suggested: "one thing simplified, or many things grouped?"). 3. Check as a class; pairs then generate one new example of each type from a video game they know. **Answers / expected outcomes:** Representational: a, c, e, g, i. Generalisation: b, d, f, h, j. The test question — "am I stripping detail from *one* model, or finding what *many* things share?" — is the transferable takeaway. **Stretch:** Identify a card that is arguably *both* (e.g. the road-network graph both simplifies one map *and* generalises "places + connections" to fit any network — social, rail, computer) and defend the dual reading.

Odd One Out: Levels of Detail (odd one out · 10 min · threes)

Resources: Board with three sets: Set A — *pixel colour values, sprite (x,y) position, player score, the brand of the player's monitor*; Set B — *bus timetable, live GPS position of every bus, route map, fare table*; Set C — *chess board state, whose turn it is, the material each piece is carved from, castling rights*. **How it runs:** 1. Trios pick the odd one out per set with the rule: justification must use the phrase "irrelevant to the problem of …" with a *named* problem. 2. Then invert: invent a different problem for which their odd one out becomes the *essential* detail and something else becomes irrelevant. 3. Share the best inversions. **Answers / expected outcomes:** A: monitor brand (irrelevant to running the game). B: live GPS of every bus (irrelevant to *planning* a journey from a timetable — but essential to a "where's my bus?" app, a perfect inversion). C: the material of the pieces (irrelevant to playing chess — essential to valuing an antique set). Students internalise that abstraction decisions are always *relative to a purpose*. **Stretch:** Trios write one exam-style sentence per set: "For the problem of ***, we can abstract away*** ___ ***because*** ."

Always, Sometimes, Never: Abstraction Claims (always-sometimes-never · 15 min · pairs then fours)

Resources: Seven statement slips: (a) "An abstraction contains less detail than the real thing"; (b) "Removing more detail makes an abstraction better"; (c) "Abstraction and decomposition are the same skill"; (d) "Two correct abstractions of the same thing can look completely different"; (e) "A model that leaves out relevant detail is a poor abstraction for that problem"; (f) "Abstraction is only used in programming"; (g) "The Tube map is wrong because distances aren't to scale". **How it runs:** 1. Pairs classify each as Always/Sometimes/Never with a one-line justification and an example. 2. Pairs merge into fours; disagreements must be settled by constructing a counter-example. 3. Teacher-led debrief targeting (b) and (c) — the two most commonly botched in exams. **Answers / expected outcomes:** (a) Always. (b) Sometimes — remove a *necessary* detail and the model stops solving the problem (canteen map with no turnings). (c) Never — abstraction *removes* detail; decomposition *breaks into sub-problems*; different skills. (d) Always possible — Tube map vs street map, different purposes. (e) Always. (f) Never — maps, timetables, models everywhere. (g) Never — it is fit for its purpose (which line/station), scale is the discarded detail. **Stretch:** Fours draft one new "Sometimes" statement whose truth hinges on the problem's purpose, and challenge another four.

2.1.2 Thinking Ahead

The Library Runner: A Human Cache (caching human demo · 20 min · whole class)

Resources: A "library" table at the far end of the room (or corridor) holding 8 fact cards face down (e.g. capital cities); a "CPU" desk at the front; one runner; a stopwatch; a small "cache" tray on the CPU desk that holds only 3 cards; a request list for the teacher where some facts repeat often (e.g. requests: France, Japan, France, Brazil, France, Japan, Egypt, France…). **How it runs:** 1. Round 1 — no cache: teacher (the program) calls out requests one at a time; the runner must jog to the library, find the card, bring it back, return it, then take the next request. Time 8 requests. 2. Round 2 — with cache: same request list, but the runner may now *keep up to 3 cards* in the desk tray. On each request, check the tray first (a "cache hit" — instant), only run on a miss. When the tray is full, the runner must choose a card to evict. Time 8 requests. 3. Compare times. Ask: why was round 2 faster? (Frequently requested data served from fast, close storage.) Which requests were slow even in round 2? (First access of each — compulsory misses.) 4. Round 3 — stale data: while the runner is mid-jog, the teacher secretly swaps the "France" card in the library for one reading "capital: Marseille (updated!)". The cached copy at the desk

still says Paris. Serve a France request from cache — the class spots the wrong/old answer. Name it: **stale data**, and discuss invalidation (when the source changes, the cached copy must be refreshed or discarded). 5. Debrief into notes: caching = storing expensive/frequently-used results in faster storage; benefit = speed, less recomputation/traffic; trade-offs = limited capacity (eviction choices) and staleness. **Answers / expected outcomes:** Round 2 measurably faster; students articulate hit vs miss, why the runner's eviction choice matters (evict the card least likely to be asked again — least recently used works well on this list), and the staleness trap. Map to real examples: web browser cache, CPU cache, DNS cache. **Stretch:** Give the runner a written eviction policy to follow (LRU vs "evict newest") and re-run — which policy wins on this request pattern, and can students craft a request list where the *other* policy wins?

Spec It Before You Build It (think-pair-share · 15 min · pairs)

Resources: One scenario per pair from a set (cinema seat-booking system, vending machine, penalty-shootout game, school attendance app); planning grid with four boxes: **Inputs / Outputs / Preconditions / What we'd cache**. **How it runs:** 1. Think (3 min, silent): individually fill the grid for the scenario. 2. Pair (5 min): merge grids; apply two audit rules — *"is each 'input' really data going in, or is it a process?"* and *"is each precondition checkable before the routine runs?"* 3. Share: pairs read one item per box; the class plays "process police", challenging anything like "validate the card number" appearing as an input (it's a process — the *card number* is the input). **Answers / expected outcomes:** For cinema booking: inputs = film choice, date/time, number of seats, seat selection, payment details; outputs = confirmation, e-ticket/reference, updated seat map; preconditions = screening exists and is in the future, seats requested ≤ seats available, user logged in; cache = the seat map of popular screenings, film listings. Equivalent quality lists for other scenarios. Key habit: inputs/outputs are *data*, not actions. **Stretch:** Pairs name one *reusable component* their system should import rather than write (payment processing, login/authentication, date picker) and give two benefits (pre-tested so fewer bugs; faster development; consistency).

Thinking-Ahead Element Sort (card sort · 15 min · threes)

Resources: Header cards — **Input**, **Output**, **Precondition**, **Caching decision**, **Reusable component**; ~18 item cards drawn from one rich scenario (an online food-delivery app), e.g. *customer's postcode*, *estimated delivery time shown to user*, *the restaurant list for a postcode searched five times tonight*, *driver must be logged in before accepting jobs*, *payment-processing module*, *basket must not be empty before checkout*, *order confirmation email*, *map/geocoding library*, *menu photos for the top 10 restaurants*, *customer's allergy notes*, *"validate the postcode"* (trap), *push notification "driver nearby"*, *login module*, *the restaurant must be open when the order is placed*, *tonight's surge-pricing multiplier* (trap — cache or not?), *rating submitted by customer*, *receipt PDF*, *basket contents*. **How it runs:** 1. Trios sort all cards under the five headers; traps force discussion. 2. Rule: every card under **Caching decision** needs a scribbled *benefit* and *risk*; every **Precondition** must be rewritten as a checkable condition. 3. Compare with a neighbouring trio; resolve differences before whole-class check. **Answers / expected outcomes:** "Validate the postcode" belongs under none of input/output — it is a process (good discussion point). Surge multiplier is a *poor* caching candidate (changes rapidly → stale fast), unlike menu photos and popular search results (expensive, frequently reused, change rarely). Preconditions emerge as Booleans: `basketSize > 0`, `restaurantOpen == TRUE`. Payment, login, maps = reusable components. **Stretch:** Trios add an invalidation rule for each cached item ("refresh restaurant list when a restaurant changes status; expire after 10 min").

Silent Debate: "Cache Everything!" (silent debate · 15 min · groups of 4)

Resources: Two A3 sheets per group: Sheet 1 — *"A news website should cache every page it ever generates."* Sheet 2 — *"A live sports-score site should cache nothing."* Different coloured pen each; silence rule. **How it runs:** 1. Half of each group starts on each sheet, writing arguments for/against in silence; arrows to connect, question marks to challenge. 2. Swap sheets at 4 minutes; students must now respond to what is written — extend, rebut, or add a real-world example — not start fresh threads only. 3. Final 3 minutes, speaking allowed: group drafts a one-sentence *policy* for each site ("cache X for Y minutes because Z"). **Answers / expected outcomes:** Sheet 1 arguments: speed and reduced server load (+) vs storage cost and stale news — a corrected story must not keep serving the old version (−). Sheet 2: fresh scores matter (+) but caching *static parts* (logos, layout, historical results) is still sensible (−) — the mature position is *selective* caching with sensible expiry, mirroring the benefit/trade-off pairing exams reward. **Stretch:** Require each policy sentence to name the invalidation trigger (time-based expiry vs event-based "when the story is edited").

2.1.3 Thinking Procedurally

Tea-Bot 3000 (decomposition precise-instructions task · 25 min · pairs, then whole class demo)

Resources: Props: kettle (empty, unplugged, or mimed), mug, teabag, spoon, milk carton (empty); teacher (or brave volunteer) as the Robot, who obeys instructions with malicious literal-mindedness and does *nothing* that is not stated; whiteboards for instruction lists. **How it runs:** 1. Brief: "Write instructions so the Robot makes a cup of tea. The Robot has never seen tea. It executes exactly what is written, in order, and halts with ERROR if an instruction is impossible or ambiguous." 2. Pairs write their instruction list (6 min). 3. Run submissions at the front. Play it brutally: "Put the teabag in the mug" before any mug is fetched → ERROR: no mug. "Boil the kettle" with no water → simulate boiling dry. "Pour the kettle" → pour where? "Add milk" → the whole carton. Let the class shout the failure each time. 4. After two or three crashed attempts, stop and name the fixes the class is already discovering: **decompose** into sub-procedures (FETCH_ITEMS, BOIL_WATER, BREW, ADD_EXTRAS), give each precise ordered steps, and note *order dependencies* (can't pour unboiled water; can't brew without the mug fetched first). 5. Pairs revise into named sub-procedures with an explicit calling order, then a final run of the best attempt should succeed. **Answers / expected outcomes:** A working answer decomposes into ~4 named sub-procedures with strict internal ordering and identified dependencies (BOIL_WATER and FETCH_ITEMS are independent — seed for 2.1.5 concurrency! — but BREW depends on both). Students articulate that decomposition = breaking a problem into smaller sub-problems, each simple enough to specify exactly, and that some steps *must* follow others because they consume earlier outputs. **Stretch:** Add parameters: rewrite BREW to take `strength` (brew time) and `milk` (TRUE/FALSE) so one sub-procedure serves every order — a reusable, generalised component (bridging back to 2.1.2 and abstraction).

Decomposition Jigsaw: Build the Game (jigsaw teaching · 25 min · home groups of 4)

Resources: A one-paragraph brief for a simple game (e.g. a top-down maze game: player moves, collects keys, avoids a patrolling guard, opens the exit); four expert task cards: **Player input & movement**, **Guard patrol logic**, **Key/door interactions**, **Score & display**. **How it runs:** 1. Each home-group member takes one expert card; expert groups convene and decompose *their* subsystem into 4–6 ordered steps, noting what data they need *from* other subsystems and what they provide *to* them. 2. Experts return home and teach their decomposition; the group assembles a master diagram showing all four subsystems and the data flowing between them. 3. Groups answer the integration question: which

subsystems could be built and tested independently, and which pairs must agree an interface first (e.g. movement must tell interactions the player's new position)? **Answers / expected outcomes:** A complete decomposition where each subsystem is a self-contained sub-problem with named inputs/outputs; students discover that decomposition creates *interfaces* between parts and that dependency, not preference, dictates build order. Movement→collision→interaction ordering per frame emerges naturally. **Stretch:** Groups mark which subsystem steps could run concurrently each frame and which cannot (guard patrol vs player input are independent; both must finish before collision checks) — pre-loading 2.1.5.

Human Sequencer: Order Matters (card sort / ordering · 15 min · groups of 5–6, kinaesthetic)

Resources: One step per A5 card for two processes, shuffled together per group. Process A (login): *display login form → user enters details → look up username → compare password hash → IF match, create session → show dashboard*. Process B (online order): *take basket contents → check stock → take payment → IF payment succeeds, decrement stock → generate confirmation → email receipt*. **How it runs:** 1. Groups first separate the two interleaved processes, then physically line up holding the cards in a defensible order (two lines). 2. Challenge round: teacher swaps two students in a line (e.g. *take payment* before *check stock*) and the group must state the concrete failure this causes, or accept the swap if it is genuinely order-independent. 3. Groups mark each adjacent pair as **must-follow** (data dependency) or **could-swap** (independent), using sticky notes. **Answers / expected outcomes:** Correct sequences with reasoning by dependency: you cannot compare a password hash before looking up the user; charging before checking stock risks paying for unavailable goods. *Generate confirmation* and *email receipt* must follow payment but "email receipt" vs "show confirmation page" are plausibly swappable — students learn ordering is forced by data dependencies, not habit. **Stretch:** Ask where a *precondition* belongs in each line (user exists; `stock >= quantity`) and have that student stand side-on as a "gate" before the step they guard.

Concept Map: One Problem, Five Lenses (concept mapping · 20 min · threes)

Resources: A3 paper; one central scenario card ("a school library self-service kiosk: scan card, borrow, return, reserve"); five colours of pen mapped to the five 2.1 skills. **How it runs:** 1. Scenario goes in the centre. In *green* (procedural), trios decompose the kiosk into sub-procedures radiating out (BORROW, RETURN, RESERVE, AUTHENTICATE…), each broken into ordered steps. 2. In other colours they annotate the same map: *blue* abstraction (details ignored — book cover art, member's height), *red* logical (decision points on branches: `IF onLoan`), *orange* ahead (preconditions, cacheable data), *purple* concurrent (steps that could overlap). 3. Trios present one branch, narrating how the five skills attack the same problem differently. **Answers / expected outcomes:** A single artefact showing decomposition as the skeleton with the other four skills as annotations — directly rehearsing the exam's favourite move, "apply computational thinking to this unseen scenario". Clear takeaway: procedural thinking gives the *structure*; the others refine it. **Stretch:** Add a sixth annotation: for each sub-procedure, name a reusable component that could implement it (barcode-scanner library, authentication module).

2.1.4 Thinking Logically

Story to Flowchart (decision-points flowchart task · 25 min · pairs)

Resources: A short narrative on paper, deliberately written as flowing prose, e.g.: *"Sam arrives at the leisure centre. Members swipe their card; if the card is valid they go straight in, but an expired card means a trip to the desk to renew first. Non-members pay on the door — unless the pool is at capacity, in which*

case they're asked to wait until a swimmer leaves. Once inside, everyone must shower before swimming. Sam swims lengths until either 40 lengths are done or the whistle signals closing time, then gets changed and leaves." Flowchart symbol key (terminator, process, decision); highlighters. **How it runs:** 1. Pairs read and highlight every point where the story *branches or repeats* — the decision points — and underline the condition driving each. 2. Rewrite each highlighted condition as an exact Boolean with both outcomes stated: `IF cardValid`, `IF poolCount < capacity`, loop `WHILE lengths < 40 AND NOT whistle`. 3. Draw the flowchart: diamonds for every decision, both exit paths labelled Yes/No, the swimming loop drawn as a backwards arrow. 4. Swap with another pair and audit with three checks: does every diamond hold a testable condition (not a vague phrase)? Does every diamond have exactly two labelled exits? Does the loop's exit condition actually terminate? **Answers / expected outcomes:** A correct chart has three decisions (member? card valid? pool at capacity?) plus a condition-controlled loop with a compound condition — students should spot that "until 40 lengths *or* whistle" becomes `WHILE lengths < 40 AND whistle == FALSE` (or exits on `lengths == 40 OR whistle` — a lovely De Morgan's discussion). Both branches of every decision are traced. This is exactly the skill of identifying decision points and how they alter flow of control. **Stretch:** Pairs add a new business rule of their own ("under-16s must be accompanied") and integrate it with minimal redrawing, then state which existing paths their new diamond affects.

Human Flowchart (role-play · 20 min · groups of ~8, kinaesthetic)

Resources: Rope or masking tape for arrows; laminated A4 signs: START, STOP, process rectangles, and blank decision diamonds with marker pens; a stack of "data tokens" — cards each describing a person at airport security (e.g. *has boarding pass: yes, bag over 100 ml liquids: no, metal detector beeps: yes*). **How it runs:** 1. The group designs airport security as a flowchart *made of themselves*: each student is a node, holding their sign; decision-students write their exact Boolean on the diamond (`hasBoardingPass == TRUE`, `liquidsOver100ml == FALSE`, `detectorBeeps == FALSE`) and point left/right for True/False. 2. Send data tokens through: a walker carries each card from START, and at each diamond the decision-student must read the card, evaluate their condition *aloud*, and physically point the walker down the correct branch. 3. Sabotage round: teacher hands in an edge-case token (missing data: "boarding pass: unknown") — the flowchart jams, and the group must add a node or rewrite a condition to handle it. 4. Debrief: what made a "good" diamond? (A condition answerable from the data on the card — testable, unambiguous, two exits.) **Answers / expected outcomes:** Students experience flow of control as literal movement: the same network of nodes sends different data down different paths. The sabotage round teaches that conditions must be evaluable for *all* inputs and that unhandled cases are design bugs, not user errors. **Stretch:** Add a loop lane (re-scan after removing belt) and ask: what guarantees this loop terminates? Introduce a `maxRescans` counter and let a student play it.

Always, Sometimes, Never: Conditions & Branches (always-sometimes-never · 15 min · pairs)

Resources: Eight statement slips: (a) "A decision point has exactly two outcomes"; (b) "`IF x >= 10` and `IF NOT (x < 10)` choose the same branch"; (c) "The ELSE branch of an IF is optional"; (d) "A WHILE loop's body executes at least once"; (e) "Changing the order of two IF statements never changes the output"; (f) "A condition must compare a variable with a value"; (g) "Every loop contains a decision point"; (h) "'If the user is old enough' is a valid condition to write in a flowchart diamond". **How it runs:** 1. Pairs sort into Always/Sometimes/Never; every verdict needs a supporting example or counter-example in pseudocode. 2. Fours compare; the teacher collects the two most contested onto the board and lets the class trace concrete values to settle them. **Answers / expected outcomes:** (a) Sometimes — IF has two, but CASE/switch can have many. (b) Always — logically equivalent. (c) Always. (d) Never guaranteed — condition may be false initially (contrast do-until, which runs at least once). (e) Sometimes — independent

IFs can swap; overlapping/elseif chains cannot (trace `x = 15` through `IF x > 10` / `IF x > 5` chains). (f) Sometimes — can compare two variables, or test a Boolean directly. (g) Always — the loop test is a decision point. (h) Never as written — must be exact: `IF age >= 18`. **Stretch:** Pairs write one statement of their own where the answer flips between "Sometimes" and "Always" depending on the language construct assumed, and defend both readings.

Odd One Out: Spot the Decision (odd one out · 10 min · threes)

Resources: Board with three sets from real scenarios: Set A — *check password matches, add item to basket, check age ≥ 18, check stock > 0*; Set B — `WHILE queue not empty`, `IF balance < price`, `x = x + 1`, `UNTIL guess == target`; Set C — *"if it's sunny", "if temperature > 25", "if raining == TRUE", "if humidity >= 80"*. **How it runs:** 1. Trios find the odd one out per set, justifying with the terms *decision point*, *condition*, *flow of control*. 2. Inversion round: rewrite the odd one so it fits the set (turn "add item to basket" into a decision: `IF stock > 0 THEN add item`; make `x = x + 1` a loop counter inside a decision; make "if it's sunny" testable: `IF sunshineHours > 6`). **Answers / expected outcomes:** A: "add item to basket" (a process; the others are decisions). B: `x = x + 1` (assignment; the others are decision points controlling flow). C: "if it's sunny" (not an exact, testable Boolean; the others are). The inversion round drills the exam habit of converting vague English into precise conditions. **Stretch:** For set B, trios classify each decision as selection or iteration and explain how each alters flow of control differently (branch once vs repeat).

2.1.5 Thinking Concurrently

Kitchen Brigade (concurrency role-play · 30 min · groups of 6–7)

Resources: Recipe card for a two-course order (e.g. soup + main: *chop vegetables, simmer soup 5 "minutes", fry chicken 4 "minutes" IN THE PAN, boil rice 3 "minutes", plate soup, plate main, garnish both*), where one "minute" = 20 real seconds on a visible timer; role badges: Head Chef (coordinator, may not cook), 4–5 Cooks, Inspector (records timings/errors); **exactly ONE frying-pan card per kitchen** — any task needing the pan requires physically holding the card; task tokens to hold while "working" (a held token = busy, cannot start another task). **How it runs:** 1. Round 1 — single core: one cook does every task in sequence while the others watch; Inspector times the full order. (This is one worker time-slicing: even if they swap between tasks, nothing truly overlaps.) 2. Round 2 — parallel: all cooks work simultaneously; Head Chef assigns tasks. Independent tasks (chop / boil rice / fry chicken) overlap and the order finishes far faster. Inspector records the new time and the *dependencies* that still forced waiting (can't plate soup before it has simmered; garnish last). 3. Round 3 — the shared pan: teacher adds a second simultaneous order that also needs frying. Two cooks need the ONE pan card. Play it straight: if one cook grabs the pan mid-way through the other's frying "minutes", the first dish is ruined — the result depends on who grabbed it when. Name it: a **race condition** on a shared resource. 4. Fix round: groups propose and test a fix — a lock (you must hold the pan card start-to-finish; others wait) or the Head Chef scheduling pan-jobs. Note the cost: waiting cooks are idle (synchronisation overhead), and a bad protocol can deadlock (demonstrate if it arises: cook A holds the pan waiting for the garnish tongs, cook B holds the tongs waiting for the pan — nobody proceeds). 5. Debrief against the spec table: benefits (faster completion, better use of many "cores", responsiveness) vs trade-offs (harder to coordinate/debug, race conditions, deadlock, synchronising results, and not always faster — plating was inherently sequential). **Answers / expected outcomes:** Round 2 beats Round 1 decisively but *not* proportionally to cook-count — dependencies and the shared pan cap the speed-up. Students can define race condition ("result depends on unpredictable timing of concurrent tasks sharing data/resources") and deadlock from lived experience, and can name which recipe tasks were parallelisable (independent) versus inherently

sequential (dependent on earlier outputs). **Stretch:** Inspector computes the theoretical minimum time (the *critical path* — longest chain of dependent tasks) and the group compares it with their actual Round 2 time; discuss why adding a seventh cook would barely help.

Parallel or Sequential? Dependency Detective (card sort · 15 min · pairs)

Resources: Task-pair cards for three scenarios. Web page load: *(download image A / download image B)*, *(fetch HTML / render the HTML)*, *(download CSS / download JS)*. Exam results day: *(print certificates / stuff envelopes with those certificates)*, *(calculate Maths grades / calculate CS grades)*, *(calculate grades / publish grades online)*. Game frame: *(update enemy AI / play background music)*, *(detect collisions / move sprites)*, *(render frame / compute next frame's physics — trick card)*. **How it runs:** 1. Pairs sort each card into **Can run concurrently** or **Must stay sequential**, writing the data dependency for every "sequential" verdict ("rendering *needs* the fetched HTML"). 2. Cross-examination: pairs swap piles and must find at least one card to challenge, forcing a dependency argument either way. 3. Debrief the trick card: physics for the *next* frame genuinely can overlap rendering the *current* one (pipelining) — dependency is between the *same* frame's stages, not adjacent frames'. **Answers / expected outcomes:** Concurrent: image A/B, CSS/JS, Maths/CS grades, AI/music, and (with the pipelining insight) render/next-physics. Sequential: fetch→render, print→stuff, calculate→publish, move→detect collisions (collision detection needs the new positions). The test question students take away: *"does one task consume the other's output?"* — check data dependencies before claiming concurrency. **Stretch:** For each "concurrent" pair, pairs state what must happen at the *end* (results synchronised/recombined — the page renders only when all assets arrive) and estimate whether concurrency actually helps on a single-core machine (time-slicing ≠ speed-up).

The Concurrency Continuum (decision continuum · 15 min · whole class)

Resources: Tape line labelled **"Definitely make it concurrent" ↔ "Keep it sequential"**; six system cards read aloud. **How it runs:** 1. Read each system; students place themselves and must justify with a benefit or trade-off from the spec when interviewed: (a) a web server handling thousands of visitor requests; (b) calculating a running total of 1,000 numbers where each step needs the previous sum; (c) a phone app downloading a file while keeping the UI responsive; (d) two threads updating one shared bank balance; (e) rendering different regions of one movie frame on a 16-core machine; (f) a program on a single-core microcontroller doing two CPU-heavy calculations. 2. Complication round: for (d), add "…protected by a lock" — who moves, and why? For (f), add "…one task is waiting on slow sensor input" — who moves? 3. Exit line: every student states one benefit *and* one trade-off before sitting down. **Answers / expected outcomes:** (a), (c), (e) cluster at concurrent (throughput; responsiveness; true parallelism on many cores). (b) sequential — each step depends on the previous output; inherently sequential. (d) sequential-leaning — race-condition risk on shared data; a lock makes concurrency safe but adds overhead (students should move partway, not fully). (f) sequential — one core means time-slicing with overhead, no true speed-up; but the complication (I/O waiting) justifies interleaving. Students rehearse the "not always faster" nuance examiners reward. **Stretch:** Challenge students at the extremes to argue the *opposite* end for 30 seconds — steelmanning the trade-off table.

Deadlock Theatre: The Two Printers (role-play · 15 min · fours, then whole class)

Resources: Two prop cards per group: **Printer** and **Scanner**; two "process" scripts: Process A — *"acquire Printer, then acquire Scanner, then photocopy (needs both), then release both"*; Process B — *"acquire Scanner, then acquire Printer, then photocopy, then release both"*; a narrator card. **How it runs:** 1. Two students play Process A and B, following their scripts literally and simultaneously, each grabbing their first resource card at the same moment; the narrator freezes the scene when both are waiting. 2.

Freeze-frame interview: "A, what are you waiting for? B, will you ever release the scanner? What happens next?" Nothing — forever. Name it: **deadlock** (each holds a resource the other needs; neither proceeds). 3. Groups have 4 minutes to script and perform a fix. Accept any that works: both processes acquire resources in the *same agreed order*; or a coordinator grants both-or-neither; or a timeout that releases and retries. 4. Class compares fixes: what does each cost (waiting/idleness, coordination overhead, retry wasted work)? **Answers / expected outcomes:** Students can narrate the four-line recipe for deadlock (two resources, two processes, opposite acquisition order, no pre-emption) and at least two prevention strategies with their costs. Connect back to the kitchen: pan + tongs was the same structure. **Stretch:** Groups extend to three processes and three resources in a cycle (circular wait) and show the ordering fix still works, then relate this to why concurrent code is "harder to design and debug" — the bug only appears with unlucky timing.